Normalização e Justificativas — Plataforma Nexo

Persistência | Aquiles Maciel Amaral

1. Objetivo

Este documento aplica e justifica a normalização do modelo relacional da plataforma Nexo, analisando as dependências funcionais consideradas e verificando a Primeira, Segunda e Terceira Formas Normais nas 25 tabelas do modelo.

2. Dependências funcionais consideradas

Em todas as 25 tabelas do modelo, a chave primária é um identificador artificial de coluna única (`id`, INT, autoincremento). Isso nos ajuda na análise de dependência funcional: como a chave tem um único atributo, toda dependência funcional observada no modelo é do tipo `id → demais atributos`, nunca uma dependência parcial de uma chave composta. A única exceção conceitual é a tabela associativa `perfil_permissao`, que tem uma chave candidata composta implícita (`id_perfil`, `id_permissao`), tratada na seção 4.

3. Primeira Forma Normal (1FN)

Uma tabela está na 1FN quando todos os seus atributos são atômicos (ou seja, sem campos multivalorados ou repetidos) e cada linha é identificável por uma chave primária.

3.1 Verificação

As 25 tabelas do modelo atendem à 1FN:

- Toda tabela possui chave primária (`id`), garantindo que cada linha seja única e identificável.
- Nenhum atributo guarda mais de um valor na mesma célula.
- Não há tabelas com grupos de colunas repetidos (por exemplo, `telefone1`, `telefone2`, `telefone3`), o que também violaria a 1FN.

3.2 Conclusão

Todas as 25 tabelas atendem à 1FN, com duas exceções controladas e documentadas (`psicologo.especialidades` e `evolucao_clinica.historico_versoes`), tratadas como desnormalização justificada na seção 6.

4. Segunda Forma Normal (2FN)

Uma tabela está na 2FN quando está na 1FN e todo atributo não chave depende da chave primária por inteiro, não apenas de parte dela. A 2FN só é uma preocupação real em tabelas com chave primária composta.

4.1 Verificação

Das 25 tabelas, 24 têm chave primária simples (`id` único), o que já garante que a 2FN automaticamente não há como existir dependência parcial quando a chave tem um único atributo.

A exceção é a tabela associativa **perfil_permissao**, cuja chave candidata natural seria composta (`id_perfil`, `id_permissao`). O modelo usa um `id` próprio como chave primária, mas mantém a chave candidata como restrição UNIQUE composta. Essa tabela tem apenas as duas chaves estrangeiras como atributos não chave, e ambas dependem da combinação completa (id_perfil, id_permissao) — na verdade, cada uma depende apenas de si mesma, e nenhum atributo descritivo adicional existe na tabela que pudesse depender de apenas metade da chave composta. Por isso, mesmo mantendo `id` como chave primária técnica, não há risco de dependência parcial.

O mesmo padrão se repete em `paciente_responsavel`-equivalente do modelo atual: a relação paciente↔responsável não usa mais tabela puramente associativa, mas sim `vinculo_responsabilidade`, que tem chave primária simples e um atributo próprio (`situacao`) que depende da linha inteira do vínculo, não de uma fração da combinação usuario+paciente.

4.2 Conclusão

Todas as 25 tabelas atendem à 2FN. Nenhum atributo não chave depende de apenas parte de uma chave composta, porque o modelo evita chaves primárias compostas e usa restrições UNIQUE para preservar as chaves candidatas naturais quando necessário.

5. Terceira Forma Normal (3FN)

Uma tabela está na 3FN quando está na 2FN e nenhum atributo não chave depende de outro atributo não chave (dependência transitiva). Esta é a forma normal mais relevante para o modelo Nexo, porque o domínio tem várias tabelas que poderiam ter sido modeladas de forma mais simples, concentrando dados que na verdade pertencem a entidades diferentes — a separação dessas entidades é o que garante a 3FN.

5.1 Casos verificados

usuario ↔ perfil_acesso. Uma modelagem menos cuidadosa poderia ter colocado o nome do perfil ("administrador", "psicologo" etc.) diretamente como um campo de texto em `usuario`. Nesse caso, outros atributos do perfil (como as permissões associadas) dependeriam do nome do perfil, não do id do usuário — uma dependência transitiva clássica (id_usuario → nome_perfil → permissões). O modelo evita isso extraindo `perfil_acesso` como tabela própria, de forma que `usuario` armazena apenas a chave estrangeira `id_perfil_acesso`, e tudo o que descreve o perfil fica isolado em sua própria tabela.

cobranca, tentativa_pagamento e pagamento. Se o modelo tivesse uma única tabela "pagamento" com todos os atributos de cobrança, tentativa e confirmação juntos, campos como `valor_confirmado` e `referencia_transacao` dependeriam da existência de uma confirmação específica, não apenas do id da cobrança — uma dependência transitiva via situação da tentativa. A separação em três tabelas (cobrança = obrigação; tentativa_pagamento = operação, confirmada ou não; pagamento = confirmação) elimina essa transitividade: cada atributo depende apenas da chave primária de sua própria tabela.

atendimento ↔ evolucao_clinica. Um atendimento pode gerar mais de um registro de evolução clínica ao longo do tempo (histórico de revisões). Se os campos de evolução (`conteudo`, `data_registro`) estivessem na própria tabela `atendimento`, o modelo não conseguiria representar mais de uma evolução por atendimento sem repetir toda a linha de atendimento — e os dados de evolução teriam uma dependência funcional em relação ao momento do registro, não apenas ao atendimento. A tabela separada `evolucao_clinica`, com `id_atendimento` como chave estrangeira, resolve isso.

compartilhamento_documento ↔ destinatario_compartilhamento ↔ registro_acesso_documento. O mesmo padrão se repete aqui: finalidade e validade do compartilhamento (que dependem só do compartilhamento em si) ficam separadas de quem é o destinatário (que pode haver mais de um) e de cada evento de acesso (que ocorre em datas diferentes e não deveria sobrescrever ou duplicar os dados do compartilhamento).

disponibilidade, bloqueio_agenda e agendamento. Poderiam ter sido modelados como uma única tabela de "agenda", mas isso faria com que o motivo de um bloqueio (atributo de bloqueio_agenda) ficasse na mesma linha que a situação de uma consulta marcada (atributo de agendamento) — dois conceitos com ciclos de vida e regras de negócio diferentes. Mantê-los separados evita dependências cruzadas entre atributos que não se relacionam diretamente.

5.2 Conclusão

Todas as 25 tabelas atendem à 3FN. O modelo evita dependências transitivas separando consistentemente entidades que representam conceitos distintos do domínio, mesmo quando isso significa mais tabelas do que uma modelagem mais simples.

6. Redundâncias eliminadas e desnormalizações justificadas

6.1 Redundâncias eliminadas

A normalização descrita nas seções 3 a 5 eliminou as seguintes redundâncias que uma modelagem menos cuidadosa poderia introduzir:

- Dados de perfil e permissão repetidos em cada linha de `usuario`, caso o nome do perfil fosse armazenado como texto livre em vez de chave estrangeira para `perfil_acesso`.
- Dados de contato e identidade duplicados entre `usuario`, `paciente` e `psicologo`, evitados ao não usar herança nem composição onde o domínio não a justifica (ver seção 1.6 do Modelo Relacional): cada tabela guarda apenas os atributos que lhe são próprios, e a ligação com `usuario` é uma referência, não uma cópia de dados.
- Repetição de dados de cobrança a cada nova tentativa de pagamento, evitada ao separar `cobranca` (a obrigação, que não muda a cada tentativa) de `tentativa_pagamento` (que referencia a cobrança por chave estrangeira, sem repetir valor, vencimento ou modalidade).
- Repetição da situação de um compartilhamento a cada novo registro de acesso, evitada ao manter `compartilhamento_documento` e `registro_acesso_documento` como tabelas separadas, ligadas por chave estrangeira.

6.2 Desnormalizações justificadas

O modelo mantém duas desnormalizações deliberadas, já identificadas na seção 3.1:

- **psicologo.especialidades**: o diagrama de classes de domínio define este atributo como multivalorado (`String [0..*]`). Criar uma tabela separada `especialidade_psicologo` (id, id_psicologo, descricao) seria a forma normalizada, mas o volume de dados por psicólogo é pequeno e a lista não precisa ser consultada, filtrada ou agregada de forma independente em nenhum caso de uso identificado até esta etapa. Por isso, optou-se por manter um único campo VARCHAR, aceitando a desnormalização em troca de simplicidade, com a ressalva de que essa decisão deve ser revista caso surjam requisitos de busca por especialidade específica.
- **evolucao_clinica.historico_versoes**: mesma lógica aplicada ao atributo `[0..*]` do diagrama de classes de domínio. Uma tabela de versionamento completa (`versao_evolucao_clinica`) normalizaria o histórico, mas até esta etapa não há caso de uso que exija consultar versões anteriores de forma estruturada — apenas preservar o histórico como registro. A estrutura normalizada pode ser adotada futuramente sem alterar as demais tabelas do modelo.

Nenhuma outra tabela do modelo apresenta desnormalização: as 23 tabelas restantes seguem estritamente a 1FN, 2FN e 3FN, sem exceções.

7. Impacto no modelo relacional

A análise de normalização não exigiu nenhuma alteração estrutural no modelo relacional: as 25 tabelas, suas colunas, chaves e relacionamentos permanecem os mesmos. O único impacto é a formalização, neste documento, de duas decisões de desnormalização que já estavam implícitas na forma como `especialidades` e `historico_versoes` foram modelados, e que agora ficam explicitamente registradas e justificadas.

8. Considerações finais

A normalização do modelo Nexo foi em grande parte, consequência direta de seguir fielmente a separação de responsabilidades definida no diagrama de classes de domínio, cada classe candidata do domínio (Usuário, Perfil de Acesso, Paciente, Psicólogo, Cobrança, Tentativa de Pagamento, Pagamento, Atendimento, Evolução Clínica, entre outras) corresponde a uma tabela própria, com responsabilidade única e sem sobreposição de atributos entre si, o mesmo princípio de coesão e baixo acoplamento que orienta a modelagem orientada a objetos do projeto se traduz no nível relacional, em um modelo já naturalmente normalizado até a 3FN, com apenas duas exceções pontuais e justificadas.